

Code Explanation

Delta Live Tables Test Suite

This test suite is designed to validate the functionality of Delta Live Tables (DLT) using PySpark. The tests cover data cleaning, data transformation, and data aggregation.

Overview

The test suite consists of a single test class `TestDeltaLiveTables` that contains four test methods. The class sets up a local Spark session and creates temporary directories for test data. The test methods cover various scenarios, including data cleaning, joining customer and order data, and aggregating customer spend.

Step 1: Setting up the Test Environment

This step involves creating a local Spark session and setting up temporary directories for test data. The Spark session is configured with Delta Lake extensions.

```
cls.spark = SparkSession.builder
    .appName("TestDeltaLiveTables")
    .master("local[*]")
    .config("spark.sql.extensions", "io.delta.sql.DeltaSparkSessionExtension")
    .config("spark.sql.catalog.spark_catalog", "org.apache.spark.sql.delta.catalog.DeltaCatalog")
    .getOrCreate()
```

Step 2: Creating Sample Data

This step involves creating sample customer and order data. The data is created as PySpark DataFrames and written to CSV files in the temporary directories.

```
customer_data = [
    ("C001", "John Doe", "john@example.com", "North"),
    ("C002", "Jane Smith", "jane@example.com", "South"),
    ("C003", "Bob Johnson", "bob@example.com", "East"),
    (None, "Invalid Customer", "invalid@example.com", "West"), # Null CustId
    ("C001", "John Doe", "john@example.com", "North") # Duplicate
]

customer_schema = StructType([
    StructField("CustId", StringType(), True),
    StructField("Name", StringType(), True),
    StructField("EmailId", StringType(), True),
    StructField("Region", StringType(), True)
])

cls.customer_df = cls.spark.createDataFrame(customer_data, schema=customer_schema)
cls.customer_df.write.csv(cls.customer_data_path, header=True)
```

Step 3: Testing Customer Data Cleaning

This step involves testing the cleaning of customer data by removing nulls and duplicates.

```
df = self.spark.read.csv(self.customer_data_path, header=True, inferSchema=True)
cleaned_df = df.filter(col("CustId").isNotNull()).dropDuplicates()
self.assertEqual(cleaned_df.count(), 3)
```

Step 4: Testing Order Data Cleaning and Total Amount Calculation

This step involves testing the cleaning of order data and calculating the total amount.

```
df = self.spark.read.csv(self.order_data_path, header=True, inferSchema=True)
cleaned_df = df.filter(col("OrderId").isNotNull()).dropDuplicates()
with_total = cleaned_df.withColumn("TotalAmount", col("PricePerUnit") * col("Qty"))
self.assertEqual(with_total.count(), 4)
```

Step 5: Testing Joining Customer and Order Data

This step involves testing the joining of customer and order data.

```
customer_df = self.spark.read.csv(self.customer_data_path, header=True, inferSchema=True)
order_df = self.spark.read.csv(self.order_data_path, header=True, inferSchema=True)
cleaned_customer = customer_df.filter(col("CustId").isNotNull()).dropDuplicates()
cleaned_order = order_df.filter(col("OrderId").isNotNull()).dropDuplicates()
order_with_total = cleaned_order.withColumn("TotalAmount", col("PricePerUnit") * col("Qty"))
joined_df = cleaned_customer.join(order_with_total, on="CustId", how="inner")
self.assertEqual(joined_df.count(), 4)
```

Step 6: Testing Customer Aggregate Spend Calculation

This step involves testing the calculation of customer aggregate spend.

```
customer_df = self.spark.read.csv(self.customer_data_path, header=True, inferSchema=True)
order_df = self.spark.read.csv(self.order_data_path, header=True, inferSchema=True)
cleaned_customer = customer_df.filter(col("CustId").isNotNull()).dropDuplicates()
```

```
cleaned_order = order_df.filter(col("OrderId").isNotNull()).dropDuplicates()
order_with_total = cleaned_order.withColumn("TotalAmount", col("PricePerUnit") * col("Qty"))
joined_df = cleaned_customer.join(order_with_total, on="CustId", how="inner")
scd_df = joined_df.withColumn("IsActive", lit(True))
                        .withColumn("StartDate", current_timestamp())
                        .withColumn("EndDate", lit(None).cast(TimestampType()))
agg_df = scd_df.groupBy("Name", "Date").sum("TotalAmount").withColumnRenamed("sum(TotalAmount)", "TotalAmount")
self.assertEqual(agg_df.count(), 3)
```